



Security Assessment

Final Report



Safe Recovery Module

August 2026

Prepared for Safe

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
System Model.....	6
Invariants and Trust Boundaries.....	6
Attack Surface.....	8
Review and Test Coverage.....	9
Known Issues.....	10
Findings Summary.....	12
Severity Matrix.....	12
Detailed Findings.....	13
Medium Severity Issues.....	15
M-01: Insufficient validation allows unfinalizable requests to permanently block account recovery.....	15
Low Severity Issues.....	18
L-01: multiConfirmRecovery rejects relayed Safe-approved EIP-1271 signatures.....	18
L-02: Recovery events can report the wrong nonce.....	20
L-03: Guardian configuration changes do not invalidate recoveries authorized under the previous configuration.....	21
L-04: Approvals collected during an active recovery remain usable after cancellation or finalization.....	23
L-05: Configuring the recovery module as a fallback handler lets any caller become a guardian.....	25
Informational Severity Issues.....	27
I-01: encodeRecoveryData uses terminology inconsistent with EIP-712.....	27
I-02: cancelRecovery and invalidateNonce expose fragmented recovery-cancellation semantics.....	28
I-03: hasGuardianApproved can report stale approvals for removed guardians.....	30
I-04: Recovery can be finalized at executeAfter although its name implies a later timestamp.....	32
I-05: Guardian recovery confirmations do not emit events.....	34
I-06: Owner and guardian disjointness is enforced only when guardians are added.....	36
I-07: Guardians cannot withdraw recovery confirmations.....	38
I-08: Safe v1.5 module guard callbacks can exploit the transient one-owner threshold during recovery.....	39
I-09: Direct confirmations can be silently rebound to a later recovery nonce.....	41
Disclaimer.....	42
About Certora.....	42

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Safe Recovery Module	https://github.com/safe-fn/dn/safe-modules/tree/recovery/v0.1.0	Audited commit: 8076191	Solidity

Project Overview

This document describes the security review of the **Safe Recovery Module**. The work was undertaken from **July 30th, 2026** to **August 5th, 2026**.

The following contracts were included in scope:

`safe-modules/modules/recovery/node_modules/candide-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol`

`safe-modules/modules/recovery/node_modules/candide-contracts/contracts/modules/social_recovery/storage/GuardianStorage.sol`

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The Safe Recovery Module project implements a guardian-based recovery module for Safe smart accounts. Safe owners configure a set of guardians and an approval threshold. Guardians can then authorize a new owner set and threshold through on-chain confirmations or EIP-712 signatures.

Once sufficient approvals are collected, anyone may initiate recovery, starting a predefined delay during which the Safe can cancel the request. After the delay, anyone may finalize recovery, and the module replaces the Safe's existing owners and applies the new threshold through its module



execution authority. Per-Safe nonces provide replay protection and allow outstanding guardian approvals to be invalidated.

Assessment Methodology

Our assessment approach combines design-level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design-level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross-protocol interactions, state-transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modeling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design-level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context-aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow-up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

System Model

The Safe Recovery Module implements guardian-based ownership recovery for Safe smart accounts. A single deployed SocialRecoveryModule maintains an independent recovery state for each Safe, keyed by the Safe address. Through owner-authorized Safe transactions, a Safe configures a sentinel-linked guardian set and a guardian approval threshold. Guardians then authorize a proposed replacement owner set and Safe threshold either through direct on-chain confirmations or through EIP-712 signatures submitted to `multiConfirmRecovery()`. Contract guardians are supported through OpenZeppelin's `SignatureChecker` and EIP-1271.

Each recovery authorization binds the target Safe, proposed owners, proposed threshold, current module nonce, chain ID, and recovery-module address. Once the configured guardian threshold is met, any account may call `executeRecovery()` to store the request, increment the Safe's recovery nonce, and begin the immutable module-wide recovery period. During that period, the Safe may cancel the request. After the deadline, any account may call `finalizeRecovery()`, which uses the module's Safe authority to remove existing owners, install the proposed owners, and apply the proposed threshold through a sequence of `execTransactionFromModule()` calls. The Safe may also invalidate its module nonce to invalidate confirmations that have not yet been converted into a scheduled request.

The module does not custody assets, charge fees, reimburse callers, or intentionally transfer value; its calls into the Safe use zero native value. Guardians, relayers, and finalizers bear their own transaction costs. Its economic authority is nevertheless equivalent to control over the Safe because a successful recovery replaces the accounts that can authorize all subsequent transfers and administrative operations. The Safe may hold arbitrary ETH, tokens, protocol positions, modules, guards, and permissions, so unauthorized owner replacement can indirectly expose all value and authority associated with the account.

Invariants and Trust Boundaries

The principal storage invariants reviewed were that each Safe's guardian list remains unique, reachable, and sentinel-terminated; the stored guardian count matches the list; and the guardian

threshold is zero exactly when no guardians exist and otherwise remains between one and the guardian count. Guardian and owner roles are intended to remain separate: a current Safe owner cannot be added as a guardian, and a current guardian cannot be installed as an owner during finalization. Because native Safe owner-management functions do not consult the recovery module, this separation is not a global invariant and must also be preserved by Safe owners and integrations.

Recovery proposals must contain a non-empty, unique, and valid owner set, a threshold between one and the number of proposed owners, and no zero, sentinel, Safe, or current guardian address. These conditions must hold before approvals are accepted and before a request is scheduled, not only during finalization. Guardian approvals must be attributable to distinct current guardians, and direct confirmations, ECDSA signatures, and EIP-1271 signatures must all authorize the same proposal and recovery round. Signatures are domain-separated by chain and module address, while the Safe address prevents one wallet's approvals from authorizing another wallet.

The nonce and guardian configuration form the recovery authorization boundary. Invalidating a nonce should make all approvals from the invalidated round unusable, and changing guardians or their threshold should not leave a request authorized under the old policy executable. Cancellation and finalization should consume the relevant request and approval round cleanly. A direct confirmation should not be silently rebound to a later nonce because of transaction ordering, and a guardian should be able to understand whether an approval remains active. Scheduled requests should remain attributable to the nonce, guardian policy, and approvals that created them.

The Safe owner quorum and guardian quorum are separate roots of authority. Safe owners are trusted to select guardians, set the guardian threshold, monitor the recovery delay, and cancel malicious requests while they retain control. A guardian threshold is trusted to replace every Safe owner after the delay and must therefore be protected as carefully as the ordinary owner threshold. Individual guardians, relayers, finalizers, and transaction submitters are otherwise untrusted. Permissionless execution and finalization are intentional, so safety cannot depend on the caller's identity.

The module's authority also depends on Safe configuration and compatibility. Safe must enforce that only enabled modules can call `execTransactionFromModule()`, while disabling the module should be understood separately from clearing recovery-module storage. The audited package is built against Safe v1.4.1, but the recovery documentation does not define an upper supported Safe version. Safe v1.4.1 does not invoke module-guard callbacks during module execution; Safe

implementations that do introduce such callbacks create an additional reentrancy boundary during owner migration. The recovery module must not also be configured as the Safe fallback handler, because fallback forwarding makes the Safe appear as msg.sender and appends the original caller to calldata, which is incompatible with the module's Safe-caller authorization assumptions.

Contract guardians and Safe dependencies introduce external-call assumptions. EIP-1271 validation may execute arbitrary guardian code and can revert or return malformed data. The module relies on the Safe's owner and module managers, linked-list conventions, self-authorized administrative calls, and return behavior. Block timestamps determine when recovery becomes finalizable, while transaction ordering determines whether cancellation, nonce invalidation, guardian changes, module re-enablement, or recovery execution takes effect first.

Attack Surface

The audit considered malicious or compromised guardians, unavailable or compromised Safe owners, untrusted relayers and finalizers, adversarial EIP-1271 guardians, transaction-ordering adversaries, and unsafe Safe configurations. Review areas included guardian setup and revocation, threshold changes, direct and aggregated confirmation, signature ordering and replay protection, nonce invalidation, request replacement, cancellation, module disablement and re-enablement, delayed finalization, and complete owner migration.

Recovery-lifecycle analysis covered malformed proposed owner sets, duplicate or reserved addresses, maximum-approval requests that cannot be replaced, stale confirmations after guardian removal or re-addition, approvals surviving guardian-policy changes, confirmations collected for the next nonce while another request is pending, and the absence of guardian-level confirmation revocation. Ordering analysis considered direct confirmations that read a later nonce at inclusion time, front-running guardian or threshold changes, and mature requests prepared while the module is disabled and finalized immediately after re-enablement.

Finalization analysis covered every transition of the Safe owner linked list and threshold, failure propagation from owner-management calls, large owner and guardian sets, transient intermediate configurations, and callbacks between module executions. In a Safe implementation that invokes an untrusted module guard or equivalent callback, the temporary threshold of one can allow reentrant Safe execution with one active owner signature. This path does not apply to the vendored Safe v1.4.1 module-execution flow, but it is relevant if Safe v1.5 or custom implementations are intended to be supported.

Configuration-composition analysis included configuring the recovery module as both an enabled module and a fallback handler. In that nonstandard configuration, arbitrary callers can reach Safe-authorized guardian-management methods through fallback forwarding and can eventually take ownership of the Safe. Monitoring and integration risks included missing confirmation events, request events and nonces that do not fully identify the underlying authorization state, stale `hasGuardianApproved()` results, fragmented cancellation semantics, and owner-guardian role overlap introduced through native Safe owner management.

There are no protocol fees, collateral, liquidation mechanics, oracle inputs, or direct DeFi integrations in scope. Economic risks arise primarily from total Safe takeover, permanent or prolonged loss of recovery availability, gas growth across owner and guardian loops, and transaction-ordering incentives around cancellation, configuration changes, and finalization. Cross-protocol review focused on Safe module and owner management, EIP-712 typed-data encoding, EIP-1271 contract signatures, OpenZeppelin `SignatureChecker` behavior, fallback forwarding, and optional module-guard callbacks.

Review and Test Coverage

Both in-scope contracts were manually reviewed: `SocialRecoveryModule` and `GuardianStorage`. The review traced every public entry point and recovery-state transition, including guardian-list mutation, threshold management, hashing and signature validation, approval accounting, nonce changes, scheduling and replacement, cancellation, finalization, and Safe owner migration. Relevant Safe v1.4.1 owner-management, module-management, fallback-handler, and signature-validation behavior was reviewed as a dependency surface, together with the OpenZeppelin signature-checking path, compiler settings, deployed bytecode notes, and the existing Certora specifications.

The vendored unit suite contains 66 scenarios covering guardian addition, removal, and threshold validation; direct and aggregated confirmations; EIP-712 encoding and EOA signature validation; signer ordering and duplicates; nonce invalidation; request execution and replacement; cancellation; disabled-module finalization; recovery timing; owner removal, replacement, and addition failures; and single-owner and multi-owner finalization. Five audit-specific scenario tests exercise unfinalizable duplicate-owner requests, stale approvals after threshold changes and guardian re-addition, owner-guardian overlap through native Safe management, and a zero recovery period. Additional findings were validated by tracing concrete state and call sequences against the Safe and module implementations.

The repository also includes existing Certora rules and invariants for guardian-list integrity, guardian counts and thresholds, authorized guardian mutation, confirmation authority, cross-wallet isolation, cancellation, nonce invalidation, disabled-module finalization, and recovery finalization. These historical specifications were reviewed as supporting evidence; no new audit-specific formal-verification campaign or stateful fuzzing was performed. Notable coverage gaps include long state-machine sequences spanning disablement and re-enablement, confirmation revocation and nonce reordering, callback-enabled Safe implementations, dual module and fallback-handler configuration, and stress testing near practical gas limits. The local test suite could not be re-executed in the review environment without resolving dependency and Node.js compatibility issues, so scenario counts describe the available test corpus rather than a fresh passing-test result.

The highest-severity issue identified is a Medium-severity recovery-availability failure: insufficient proposal validation allows guardians to schedule an unfinalizable request, and a request approved by every guardian cannot be replaced with a valid request when the existing Safe owners are unavailable to cancel it. The Low-severity findings primarily concern authorization state surviving guardian changes, cancellation, or finalization; incomplete EIP-1271 support; incorrect event attribution; and unsafe fallback-handler composition. Informational findings highlight nonce rebinding, irrevocable confirmations, conditional callback risk during the transient one-owner threshold, incomplete monitoring signals, and owner-guardian role overlap. Module disablement is addressed separately as known issue K-O2. The central security takeaway is that recovery authorization must be validated before scheduling, bound to an explicit nonce and guardian-policy epoch, invalidated consistently across every lifecycle transition, and applied to the Safe atomically under clearly documented version and configuration assumptions.

Known Issues

The issues below were known by the review cutoff date, August 5, 2026. A known issue is a reportable behavior affecting the scoped Safe Recovery Module that had already been documented publicly or disclosed by the client. Known issues are excluded from the novel-finding total.

Some of these issues were independently identified during the review before comparison with the prior disclosures. These are marked as rediscovered below. This designation records review coverage; it does not claim novelty or duplicate credit. A match requires the same underlying behavior and impact, not merely a related technical concept.

Audit references use the following identifiers: [A24 – Ackee Social Recovery Module v0.1.0](#), and [FV-5 – Actions after disabling Recovery Module](#).

ID	Client-confirmed known issue	Prior disclosure	Rediscovered
K-01	Confirmed guardian approvals remain usable after guardian removal and re-addition until the Safe invalidates the recovery nonce	A24-W1	Yes
K-02	Previously configured guardians can confirm and schedule a recovery while the module is disabled	FV-5	Yes

In total, the known set contains **2 issues**, both of which were independently rediscovered during the review.

K-01 reflects the residual behavior documented by Ackee W1. Ackee marked its finding as fixed after `invalidateNonce()` was introduced, but invalidation remains an explicit Safe action rather than an automatic consequence of guardian removal or reconfiguration. Confirmations therefore continue to persist when the Safe does not invoke that mitigation.

K-02 matches the broad intention documented in FV-5 that disabling the module should prevent recovery activity at every stage, including recovery initiation. The formal-verification issue's simplified expected outcome and implemented rule focused on preventing finalization while disabled. This review independently identified the remaining lifecycle consequence: guardians can approve and schedule a request while the module is disabled, allow its delay to expire, and finalize it immediately if the same module is later re-enabled without atomic cleanup.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	1	1	0
Low	5	5	0
Informational	9	9	0
Total	15	15	0

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
M-01	Insufficient validation allows unfinalizable requests to permanently block account recovery	Medium	Acknowledged
L-01	multiConfirmRecovery rejects relayed Safe-approved EIP-1271 signatures	Low	Acknowledged
L-02	Recovery events can report the wrong nonce	Low	Acknowledged
L-03	Guardian configuration changes do not invalidate recoveries authorized under the previous configuration	Low	Acknowledged
L-04	Approvals collected during an active recovery remain usable after cancellation or finalization	Low	Acknowledged
L-05	Configuring the recovery module as a fallback handler lets any caller become a guardian	Low	Acknowledged
I-01	encodeRecoveryData uses terminology inconsistent with EIP-712	Informational	Acknowledged
I-02	cancelRecovery and invalidateNonce expose fragmented recovery-cancellation semantics	Informational	Acknowledged
I-03	hasGuardianApproved can report stale approvals for removed guardians	Informational	Acknowledged

I-04	Recovery can be finalized at <code>`executeAfter`</code> although its name implies a later timestamp	Informational	Acknowledged
I-05	Guardian recovery confirmations do not emit events	Informational	Acknowledged
I-06	Owner and guardian disjointness is enforced only when guardians are added	Informational	Acknowledged
I-07	Guardians cannot withdraw recovery confirmations	Informational	Acknowledged
I-08	Safe v1.5 module guard callbacks can exploit the transient one-owner threshold during recovery	Informational	Acknowledged
I-09	Direct confirmations can be silently rebound to a later recovery nonce	Informational	Acknowledged

Medium Severity Issues

M-01: Insufficient validation allows unfinalizable requests to permanently block account recovery

Severity: Medium	Impact: High	Likelihood: Low
Files: SocialRecoveryModule.sol	Status: Acknowledged	

Description:

The module can schedule recovery requests that `finalizeRecovery` will always reject. If every guardian approved such a request, the guardians cannot replace it with a valid one. Only the existing Safe owners can cancel it. Recovery can therefore become permanently blocked when those owners are inaccessible.

`confirmRecovery` and `multiConfirmRecovery` only check that the owner array is non-empty and the new threshold is valid:

```
require(_newOwners.length > 0, "SM: owners cannot be empty");
require(_newThreshold > 0 && _newOwners.length >= _newThreshold, "SM: invalid new threshold");
```

In contrast, the check that prevents a guardian from becoming an owner is only performed during finalization:

```
require(!isGuardian(_wallet, newOwners[i]), "SM: new owner cannot be guardian");
```

The project's [account-recovery documentation](#) presents family members and close friends as recovery contacts and describes two-of-three as a typical guardian setup. It does not state that a recovery contact cannot become a new owner. Appointing a guardian as an owner is therefore a plausible mistake during recovery.

For example, consider a single-owner Safe whose owner has died. The Safe has three family members as guardians and a guardian threshold of two:

1. The family members agree that one of the guardians should become the new Safe owner.
2. All three guardians approve the request. The module accepts it and stores an approval count of three.
3. After the recovery period, `finalizeRecovery` reverts because the proposed owner is a guardian. The revert also restores the pending request.
4. The guardians approve a corrected request. Replacing the pending request requires more than its three stored approvals, but only three guardians exist. The replacement cannot be scheduled.

`invalidateNonce` does not clear a scheduled request. The only remaining escape hatch is `cancelRecovery`, which must be called by the Safe. In this example, the sole owner is unavailable, so the request cannot be cleared.

The same problem affects duplicate owners, the zero address, the owner sentinel, and the Safe itself. These values pass the confirmation and scheduling paths but fail during finalization.

The impact is high because the Safe may remain inaccessible when recovery is needed. No malicious guardian is required. However, permanent blocking requires the invalid request to have the maximum possible approval count. This happens whenever the guardian threshold equals the guardian count.

Recommendations:

Validate the proposed owner set before recording confirmations or scheduling a recovery. Reject current guardians, duplicates, the zero address, the owner sentinel, and the Safe itself. Apply the validation in both confirmation paths and again when scheduling.

Also provide a way to clear a request that becomes unfinalizable. This could be a guardian-authorized cancellation mechanism or request expiry that does not require access to the existing Safe owners.

Customer Response:

Acknowledged. However, this is something we don't see as requiring immediate fix under our security assumptions (i.e. guardians will only provide valid inputs). While the module cannot determine whether such input is accidental or malicious, certain proposals can be accepted and scheduled even though they cannot be finalized. The Candide team will be advised to add validation for obviously unfinalizable proposals.

Low Severity Issues

L-01: multiConfirmRecovery rejects relayed Safe-approved EIP-1271 signatures

Severity: Low	Impact: Low	Likelihood: Low
Files: SocialRecoveryModule.sol	Status: Acknowledged	

Description:

`multiConfirmRecovery` treats every empty signature as a direct confirmation and requires the signer to be `msg.sender`. This prevents a relay from submitting an empty EIP-1271 signature for a Safe guardian, even when the Safe has approved the recovery hash in `signedMessages`. Safe validates these approved messages only when the signature is empty, so using a non-empty signature is not a workaround.

`CompatibilityFallbackHandler`'s `isValidSignature` checks `signedMessages` when the signature is empty:

```
function isValidSignature(bytes calldata _data, bytes calldata _signature)
public view override returns (bytes4) {
    ...
    if (_signature.length == 0) {
        require(safe.signedMessages(messageHash) != 0, "Hash not approved");
    }
    ...
}
```

As a result, an approved recovery confirmation from a Safe guardian cannot be relayed or combined with other guardian signatures in `multiConfirmRecovery`.

Recommendations:

Treat an empty signature as a direct confirmation only when the signer is also `msg.sender`. Otherwise, pass the signature, including an empty one, to `validateGuardianSignature` so the guardian contract can validate it through EIP-1271.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

L-02: Recovery events can report the wrong nonce

Severity: Low	Impact: Low	Likelihood: Low
Files: SocialRecoveryModule.sol	Status: Acknowledged	

Description:

When a recovery is executed, the module increments the wallet nonce but does not store the nonce in the recovery request. `cancelRecovery` later reports the cancelled recovery as `walletsNonces[msg.sender] - 1`.

If the Safe calls `invalidateNonce` while the recovery is pending, the wallet nonce is incremented again. A later cancellation deletes the correct recovery request but emits `RecoveryCanceled` with the invalidated nonce instead of the nonce used to execute the recovery. Off-chain systems can therefore associate the cancellation with the wrong recovery and continue treating the cancelled recovery as active.

The same issue affects the `RecoveryCanceled` event emitted when replacing a recovery and the `RecoveryFinalized` event.

Recommendations:

Store the nonce used to execute a recovery in `RecoveryRequest` and use the stored value when emitting cancellation and finalization events. Also prevent `invalidateNonce` from being called while an executed recovery is pending.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

L-03: Guardian configuration changes do not invalidate recoveries authorized under the previous configuration

Severity: Low	Impact: High	Likelihood: Low
Files: GuardianStorage.sol SocialRecoveryModule.sol	Status: Acknowledged	

Description:

Guardian configuration changes do not cancel a recovery authorized under the previous configuration. A scheduled recovery remains finalizable even if the Safe later revokes guardians or increases the guardian threshold.

The configuration functions `addGuardianWithThreshold`, `revokeGuardianWithThreshold` and `changeThreshold` only update the guardian set or threshold.

Once a request is scheduled, `finalizeRecovery` checks that the delay has passed but does not verify that the request is still authorized by the current guardian configuration:

```
RecoveryRequest storage request = recoveryRequests[_wallet];
require(uint64(block.timestamp) >= request.executeAfter, "SM: recovery period
still pending");
```

Consider the following scenario:

1. A Safe has 5 guardians and a guardian threshold of 2.
2. The owner discovers that 2 guardian accounts are compromised and submits `changeThreshold(3)`.
3. The compromised guardians frontrun the threshold change and schedule a malicious recovery while the threshold is still set to 2.
4. The owner's transaction executes and raises the threshold to three, but the malicious recovery remains scheduled.

5. After the recovery delay, the attackers call `finalizeRecovery` and take ownership of the Safe unless the owner notices the request and separately calls `cancelRecovery`.

Successful exploitation results in complete loss of control over the affected Safe. The likelihood is low because the attacker must already control the old guardian threshold, and the active owner has the full recovery delay to notice and cancel the request. However, the threshold change can appear to remediate the compromise while leaving the malicious recovery executable.

Recommendations:

Treat every successful guardian-set or guardian-threshold change as a new recovery authorization epoch. After a configuration change, atomically delete any scheduled recovery and increment the Safe's recovery nonce to invalidate approvals collected under the previous configuration. This logic should be centralized in an internal `_cancelRecovery(address wallet)` function and called once after each successful configuration mutation.

As defense in depth, store the guardian-configuration epoch in each scheduled request and verify it again during finalization.

Customer Response:

Acknowledged. However, the owner is expected to use a notification service alongside the module to call `cancelRecovery()` or `invalidateNonce()`, depending on the recovery lifecycle, when such a situation occurs.

L-04: Approvals collected during an active recovery remain usable after cancellation or finalization

Severity: Low	Impact: Low	Likelihood: Low
Files: SocialRecoveryModule.sol	Status: Acknowledged	

Description:

When a recovery request is started, `_executeRecovery()` increments the wallet nonce after storing the request:

```
recoveryRequests[_wallet] = RecoveryRequest(_approvalCount, _newThreshold,
executeAfter, _newOwners);
walletsNonces[_wallet]++;
```

The active request therefore corresponds to nonce `N`, while new guardian confirmations are immediately accepted for nonce `N + 1`. Neither `confirmRecovery()` nor `multiConfirmRecovery()` prevents guardians from collecting these next-round approvals while the first request is pending.

Both paths that terminate the active request delete it without consuming the current nonce:

```
function finalizeRecovery(address _wallet) external whenRecovery(_wallet) {
    // ...
    delete recoveryRequests[_wallet];
    // Owner migration...
}
// ...
function cancelRecovery() external whenRecovery(msg.sender) {
    delete recoveryRequests[msg.sender];
    emit RecoveryCanceled(msg.sender, walletsNonces[msg.sender] - 1);
}
```

Consequently, approvals collected for nonce $N + 1$ remain valid after the request for nonce N is canceled or finalized. Because `executeRecovery()` is permissionless, any account can use a sufficiently approved proposal to start another recovery immediately, without any new guardian action.

For example, while a legitimate recovery is waiting for its delay, compromised guardians can approve a second proposal under the already-incremented nonce. Once the first recovery is finalized, the second proposal can be submitted immediately. The newly installed owners inherit a Safe for which another recovery has already been fully approved. The second request still observes its configured recovery delay, and the Safe can invalidate the nonce defensively, but neither protection prevents the unexpected request from being pre-staged.

The same behavior applies after cancellation: canceling the pending request does not invalidate approvals already collected for the current nonce.

Recommendations:

Consume the current approval round when a request is canceled or finalized by incrementing `walletsNonces[_wallet]`. Store the nonce associated with each `RecoveryRequest` so cancellation and finalization events can continue reporting the correct request nonce after this change.

Customer Response:

Acknowledged. However, as with L-03, such malicious activity should prompt the owners to cancel the recovery and replace the compromised guardians.

L-05: Configuring the recovery module as a fallback handler lets any caller become a guardian

Severity: Low	Impact: High	Likelihood: Low
Files: GuardianStorage.sol FallbackManager.sol	Status: Acknowledged	

Description:

If `SocialRecoveryModule` is configured as both an enabled module and the Safe fallback handler, any external account can add itself as a guardian. The attacker can then schedule a recovery and replace the Safe owners.

Safe forwards unknown function calls to its fallback handler and appends the original caller to the calldata. The handler nevertheless observes the Safe itself as `msg.sender`. An attacker can therefore call the Safe with calldata for:

```
addGuardianWithThreshold(attacker, 1)
```

The Safe forwards the call to the recovery module. Because `msg.sender` is the Safe, `onlyWhenModuleIsEnabled()` succeeds. Solidity accepts the extra 20 bytes appended by the fallback manager, and the attacker becomes a guardian with a threshold of one.

The attacker can then confirm and schedule a recovery naming an attacker-controlled owner. Once the recovery period expires, the enabled module can replace the existing owners.

This results in a complete wallet takeover, but requires the recovery module to be incorrectly configured as the fallback handler. The module has no intended fallback-handler functionality, so the likelihood is low and primarily concerns deployment or integration mistakes.

Recommendations:

Ensure Safe-only guardian-management functions reject calls forwarded by the fallback manager. One simple defense is to enforce the canonical calldata length for their static arguments, since fallback forwarding appends 20 bytes. More generally, use an authorization mechanism that distinguishes an owner-authorized Safe call from fallback forwarding.

Customer Response:

Acknowledged

Informational Severity Issues

I-01: encodeRecoveryData uses terminology inconsistent with EIP-712

Description:

EIP-712 uses `encodeData` to refer to the encoding of a struct's member values that is subsequently combined with the type hash to derive the struct hash. However, `encodeRecoveryData` returns the complete EIP-191/EIP-712 signing preimage, `\x19\x01 || domainSeparator || recoveryHash`. Although the implementation is correct, the function name may lead integrators or reviewers to mistake its return value for the inner EIP-712 struct encoding, increasing the likelihood of integration mistakes such as hashing at the wrong stage.

Recommendations:

Rename the function to `encodeRecoverySignableData` to better describe its return value.

If preserving the existing public ABI is required, retain `encodeRecoveryData` as a deprecated wrapper around the newly named function.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-02: `cancelRecovery` and `invalidateNonce` expose fragmented recovery-cancellation semantics

Description:

The module exposes two cancellation functions, but neither stops all recovery activity. `cancelRecovery` deletes a scheduled request, while `invalidateNonce` invalidates approvals collected for the current nonce.

```
function cancelRecovery() external whenRecovery(msg.sender) {
    delete recoveryRequests[msg.sender];
    // ...
}

function invalidateNonce() external {
    walletsNonces[msg.sender]++;
    // ...
}
```

These operations address different phases of the same recovery lifecycle. If a recovery is scheduled while guardians collect approvals for a replacement, `cancelRecovery` preserves the replacement approvals and `invalidateNonce` preserves the scheduled recovery. A Safe owner must call both functions to stop all recovery activity. This makes the emergency flow harder to understand and increases the risk of incomplete integrations.

Recommendations:

Expose a single `cancelRecovery` function that deletes any scheduled request and increments the Safe's recovery nonce. This would cancel the active request and invalidate all approvals collected for the current nonce in one transaction.

If granular controls are still needed, expose them under explicit names such as `cancelScheduledRecovery` and `invalidatePendingApprovals`.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-03: hasGuardianApproved can report stale approvals for removed guardians

Files:

modules/recovery/node_modules/candidate-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol

Description:

`hasGuardianApproved()` returns the raw approval value stored in `confirmedHashes` without verifying that `_guardian` is currently a guardian of `_wallet`.

```
function hasGuardianApproved(
    address _wallet,
    address _guardian,
    address[] calldata _newOwners,
    uint256 _newThreshold
) public view returns (bool) {
    uint256 _nonce = nonce(_wallet);
    bytes32 recoveryHash = getRecoveryHash(
        _wallet,
        _newOwners,
        _newThreshold,
        _nonce
    );
    return confirmedHashes[recoveryHash][_guardian];
}
```

An address can only set this value while it is an authorized guardian because the recovery confirmation functions validate guardian membership. However, removing the guardian does not clear its recorded approvals or advance the recovery nonce.

Consequently, the function can return `true` for an address that is no longer a guardian. This differs from `getRecoveryApprovals()`, which only counts approvals belonging to guardians in the wallet's current guardian list.

For example:

1. Guardian `G` confirms a recovery request.

2. The Safe removes **G** from its guardian set.
3. A UI or third-party integration calls `hasGuardianApproved()` for **G**.
4. The function returns `true`, even though **G** is no longer an active guardian and its approval is not currently counted toward recovery execution.

This may cause interfaces and integrations to display an inaccurate guardian approval state or make incorrect decisions when determining which active guardians have approved a recovery. While the inconsistency does not directly affect the module's approval count, it makes the public query API ambiguous and potentially misleading.

Recommendations:

Consider checking current guardian membership before returning the approval:

```
return isGuardian(_wallet, _guardian) &&  
    confirmedHashes[recoveryHash][_guardian];
```

Alternatively, rename or document the function to clarify that it reports whether the address recorded an approval at the current nonce, regardless of its current guardian status. Automatically invalidating the recovery nonce when the guardian configuration changes would also prevent stale approvals from persisting across configuration updates.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-04: Recovery can be finalized at `executeAfter` although its name implies a later timestamp

Files:

modules/recovery/node_modules/candide-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol

Description:

When a recovery reaches the required guardian threshold, `_executeRecovery()` calculates `executeAfter` as the current timestamp plus the configured recovery period:

```
function _executeRecovery(address _wallet, address[] calldata _newOwners, uint256 _newThreshold, uint256 _approvalCount) internal {
    // ...
    uint64 executeAfter = uint64(block.timestamp + recoveryPeriod);
    recoveryRequests[_wallet] = RecoveryRequest(_approvalCount, _newThreshold, executeAfter, _newOwners);
    // ...
}
```

`finalizeRecovery()` subsequently permits finalization when the current timestamp is greater than or equal to that value:

```
function finalizeRecovery(address _wallet) external whenRecovery(_wallet) {
    RecoveryRequest storage request = recoveryRequests[_wallet];
    require(uint64(block.timestamp) >= request.executeAfter, "SM: recovery period still pending");
    // ...
}
```

The name `executeAfter` suggests that recovery can be finalized only after the stored timestamp. However, the inclusive comparison also allows finalization when `block.timestamp` is exactly equal to `executeAfter`.

Recommendations:

Either change the comparison to `block.timestamp > request.executeAfter` to enforce strict-after semantics, or rename `executeAfter` to a name such as `executableAt` to communicate that the boundary is inclusive.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-05: Guardian recovery confirmations do not emit events

Files:

modules/recovery/node_modules/candide-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol

Description:

`confirmRecovery()` and `multiConfirmRecovery()` record guardian approvals in `confirmedHashes` without emitting corresponding events:

```
function confirmRecovery(address _wallet, address[] calldata _newOwners,
uint256 _newThreshold, bool _execute) external {
    // ...
    confirmedHashes[recoveryHash][msg.sender] = true;
    // ...

function multiConfirmRecovery(
    address _wallet,
    address[] calldata _newOwners,
    uint256 _newThreshold,
    SignatureData[] memory _signatures,
    bool _execute
) external {
    // ...
    confirmedHashes[recoveryHash][value.signer] = true;
    // ...
```

Consequently, systems that monitor the module through event subscriptions receive no notification as guardian approvals accumulate. They must instead inspect and decode every successful transaction sent to these functions. This makes it more difficult for wallet interfaces and monitoring services to notify Safe owners and guardians that a recovery proposal is approaching its approval threshold.

Recommendations:

Emit an event whenever a confirmation is recorded, including the wallet, guardian, recovery hash, and nonce. `multiConfirmRecovery()` should emit one event for each guardian whose confirmation is recorded.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-06: Owner and guardian disjointness is enforced only when guardians are added

Files:

modules/recovery/node_modules/candide-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol

Description:

The module attempts to keep Safe owners and recovery guardians as separate roles. `addGuardianWithThreshold()` rejects a proposed guardian that is already an owner, and `finalizeRecovery()` rejects a proposed owner that is currently a guardian.

```
// When adding a guardian:
require(
    !ISafe(payable(msg.sender)).isOwner(_guardian),
    "GS: guardian cannot be an owner"
);

// During finalization:
require(
    !isGuardian(_wallet, newOwners[i]),
    "SM: new owner cannot be guardian"
);
```

However, the invariant is not enforced when the Safe modifies its owner set. An owner-authorized Safe transaction can call `addOwnerWithThreshold()` or `swapOwner()` with an address that is already a guardian. The Safe has no knowledge of the recovery module's guardian set, so the operation succeeds and the address simultaneously becomes an owner and a guardian.

This is primarily a configuration and integration risk, it breaks the role-separation assumption that the module otherwise tries to enforce.

Safe owners may unintentionally reduce the independence of the recovery policy by promoting existing guardians to owners. Compromise of an overlapping key then affects both ordinary wallet control and social recovery.

Recommendations:

Document that a Safe must not add an existing guardian as an owner and expose a view function that allows integrations to detect role overlap.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-07: Guardians cannot withdraw recovery confirmations

Files:

modules/recovery/node_modules/candide-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol

Description:

Once a guardian confirms a recovery, they cannot withdraw that confirmation. This leaves no way to correct a mistake or reject a request that is later found to be malicious.

Both confirmation paths only set approvals to **true**:

```
confirmedHashes[recoveryHash][guardian] = true;
```

The approval remains valid until the Safe advances its recovery nonce and can be combined with later confirmations to reach the guardian threshold. Only the Safe can invalidate the nonce. This may not be available when recovery is needed because the existing owners have lost access.

The issue is most relevant when a guardian was temporarily compromised, approved the wrong owner set, or later changes their decision after receiving new information.

Recommendations:

Allow a guardian to revoke their confirmation for a specific recovery hash and nonce. If confirmations may also be withdrawn after scheduling, cancel the request when its remaining approval count falls below the guardian threshold.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-08: Safe v1.5 module guard callbacks can exploit the transient one-owner threshold during recovery

Files:

modules/recovery/node_modules/candide-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol

safe-smart-account/contracts/base/ModuleManager.sol

Description:

When the module is used with Safe v1.5, `finalizeRecovery()` exposes a temporary threshold of one to module guard callbacks. A malicious or compromised guard can use this state to execute a Safe transaction with one owner signature.

The function changes owners through separate module transactions. Each removal or addition sets the threshold to one, and the requested threshold is restored only after all owner changes:

```
removeOwner(previousOwner, owner, 1);  
addOwnerWithThreshold(newOwner, 1);  
// ...  
changeThreshold(newThreshold);
```

Safe v1.5 calls `checkModuleTransaction()` before each module transaction and `checkAfterModuleExecution()` afterward. These callbacks can therefore observe the Safe while its threshold is one.

If the guard has a valid signature from an owner present at that point, it can reenter `Safe.execTransaction()` and execute an arbitrary transaction with that single signature. This bypasses both the original and intended recovery thresholds.

The scenario requires Safe v1.5, a malicious or compromised module guard, and one relevant owner signature during finalization. This makes exploitation unlikely. Safe v1.4.1 is not affected because it does not support module guards.

Recommendations:

Perform the complete owner migration within one guarded module transaction so callbacks only observe the state before or after migration. Otherwise, reject finalization when a module guard is configured and document the compatibility restriction.

Customer Response:

Acknowledged. Will be suggested to the Candide Team for future iteration.

I-09: Direct confirmations can be silently rebound to a later recovery nonce

Files:

modules/recovery/node_modules/candidate-contracts/contracts/modules/social_recovery/SocialRecoveryModule.sol

Description:

The direct confirmation entry points do not receive an expected nonce. They read it from storage when the transaction is included, so a pending confirmation can be recorded in a later recovery round than the guardian intended.

For example, a guardian broadcasts `confirmRecovery()` while the nonce is `n`. Before the transaction is included, the Safe calls `invalidateNonce()` and advances it to `n + 1`. The guardian transaction does not revert. It reads `n + 1` and records the approval for the new round.

This weakens `invalidateNonce()` as a way to clear pending approvals. Transaction reordering or delayed inclusion can cause an approval intended for the invalidated round to survive in the next one.

The empty-signature path in `multiConfirmRecovery()` has the same issue because `msg.sender` confirms directly. Non-empty signatures are already bound to the nonce through the signed recovery hash and fail if it changes.

The guardian must still be valid and the threshold must still be reached. The issue is limited to an approval being applied to an unintended round.

Recommendations:

Add an `_expectedNonce` parameter and require it to match `walletsNonces[_wallet]` before recording a direct confirmation. Alternatively, accept an expected recovery hash and require it to match the hash computed from the current state.

Customer Response:

Acknowledged. Will be suggested to the Candidate Team for future iteration.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.